

Reviewer Pack — Minimal Local Indeterminacy in Noncommutative Geometry and R...

Entry 239fea861c6e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 01:25:53 UTC

Minimal Local Indeterminacy in Noncommutative Geometry and Resolution Proof Width

Entry ID: 239fea861c6e **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 01:25:53 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative Geometry **Field B** (complexity object): Resolution Proof Complexity

Statement:

For every d -regular graph G , the minimal local indeterminacy ($\text{mli}(G)$) of its associated noncommutative algebraic structure is linearly correlated with its resolution proof width $w(\varphi_G)$, such that $\text{mli}(G) = \Theta(w(\varphi_G))$.

Rationale (proposer's reasoning):

The use of noncommutative geometry in modeling complex systems could reveal hidden structures within the resolution proof process, potentially leading to new complexity lower bounds. The local indeterminacy measures how much a space deviates from commutativity, which might correlate with the branching factor in resolution proofs.

Taxonomy category: NoncommutativeGeometry_ResolutionProofComplexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 56942881fef2a27c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient between minimal local indeterminacy ($\text{mli}(G)$) and resolution proof width ($w(\varphi_G)$) for all d -regular graphs G with $n \leq 40$ variables is ≥ 0.8 , and there are no seeds where $\text{mli}(G) > w(\varphi_G)$. Falsified if either the correlation coefficient < 0.8 or any seed produces an $\text{mli}(G) > w(\varphi_G)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_d_regular_graph(n, d):
        if (n * d) % 2 != 0 or d > n - 1:
            return None
        graph = {i: [] for i in range(n)}
        edges = set()
        while len(edges) < (n * d) // 2:
            u, v = random.sample(range(n), 2)
            if u == v or (u, v) in edges or (v, u) in edges:
                continue
            graph[u].append(v)
            graph[v].append(u)
            edges.add((u, v))
        return graph

    def calculate_mli(graph):
        n = len(graph)
        mli = 0
        for i in range(n):
            for j in range(i + 1, n):
                if j not in graph[i]:
                    mli += 1
        return mli

    def resolution_width(cnf):
        # Simplified version of resolution width calculation
        queue = cnf[:]
        resolvents = set()
        while queue:
            clause = queue.pop(0)
            for other_clause in queue:
```

```

        if any(abs(lit) == abs(other_lit) for lit, other_lit in zip(clause, other_clause)):
            new_clause = [lit for lit in clause + other_clause if abs(lit) != abs(other_lit)]
            new_clause.sort()
            resolvent = tuple(new_clause)
            if resolvent not in resolvents:
                resolvents.add(resolvent)
                queue.append(list(resolvent))
    return len(resolvents)

def generate_cnf(n):
    cnf = []
    for i in range(1, n + 1):
        clause = [-i, -(-i % (n // 2) + 1)]
        cnf.append(clause)
    return cnf

n = random.randint(5, 40)
d = random.randint(2, min(n - 1, 3))
graph = generate_d_regular_graph(n, d)
if not graph:
    return {
        "metric_name": "mli(G)",
        "metric_value": None,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

mli = calculate_mli(graph)
cnf = generate_cnf(n)
width = resolution_width(cnf)

return {
    "metric_name": "mli(G)",
    "metric_value": mli,
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": mli <= width,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all(result["conjecture_holds"] for result in results):
        mean_value = sum(result["metric_value"] for result in results) / len(results)
        std_dev = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) / len(results))
        support_fraction = len([result for result in results if result["conjecture_holds"]]) / len(results)

```

```

        print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mli(G) > width\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TRIAL: {'metric_name': 'mli(G)', 'metric_value': None, 'instances_tested': 1, 'n_max': 33, 'conjecture_holds': False}

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_43da3aef.py", line 99, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_43da3aef.py", line 82, in run_trial
    width = resolution_width(cnf)
           ^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_43da3aef.py", line 52, in resolution_width
    new_clause = [lit for lit in clause + other_clause if abs(lit) != abs(other_lit)]
                                                         ^^^^^^^^^

```

NameError: name 'other_lit' is not defined

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the conjecture's support or falsification. | next: Review and debug the test code to ensure it can run successfully and produce the required data for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13754
2	preregistration	ollama_remote	glm4:latest	0	0	20383
3	novelty	ollama_remote	glm4:latest	0	0	15070
4	novelty	ollama_remote	glm4:latest	0	0	26520
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15220
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	25557

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16806
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15125
9	judge	ollama_remote	glm4:latest	0	0	13301

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 161738 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/239fea861c6e.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/239fea861c6e.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/239fea861c6e.tar.gz (if generated)